mecaniQA-api

OAT 2 - Padrão de Projeto e Diagrama de Atividade

Equipe Vitória - 28/09/2026

Agenda

1. Contexto
2. Ordem de Serviço e Pedido de Peças
3. Entidade associativa
4. Padrão Builder
5. DTOs e Mappers
6. Rotas
7. Diagramas

Contexto

Na OAT 1 a gente fez o CRUD de Peça e Serviço. Nessa etapa a oficina precisava controlar as ordens de serviço e os pedidos de peças.

- Os dados continuam em memória, nos repositories Singleton da OAT 1
- Não usamos Lombok, então construtores, getters e setters foram feitos na mão
- Criamos as classes OrdemServico e PedidoPecas
- A OS é construída usando o padrão Builder
- Os controllers agora trabalham com DTO e não com a entidade

Ordem de Serviço e Pedido de Peças

`OrdemServico`

- `codigoUnico, nomeCliente`
- `placaVeiculo, descricaoProblema`
- `servicos (List<Servico>)`
- `valorTotal`
- `dataAbertura, dataUltimaAtualizacao`
- `status`

Status:
`ABERTO, PENDENTE_PAGAMENTO, PAGO, EM_EXECUCAO, EXECUTADO`

`PedidoPecas`

- `codigoUnico, fornecedor`
- `itens (List<ItemPedido>)`
- `dataCriacao, dataUltimaAtualizacao`
- `status`

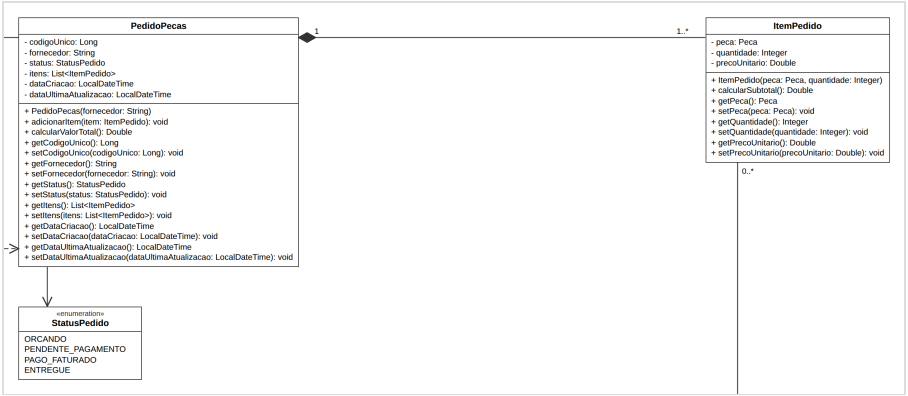
Status:
`ORCANDO, PENDENTE_PAGAMENTO, PAGO_FATURADO, ENTREGUE`

Os status foram feitos como enum, igual fizemos com a CategoriaPeca na OAT 1.

Entidade associativa (ItemPedido)

Um pedido tem várias peças e cada peça pode estar em vários pedidos. A quantidade não dava pra ficar na Peca nem no Pedido, então criamos a classe ItemPedido no meio.

- ItemPedido tem a peca, a quantidade e o precoUnitario
- O PedidoPecas tem uma lista de ItemPedido (composição, 1 para 1..*)
- Cada item aponta para uma Peca
- Na US04 o controller chama o adicionarItem() do pedido



Padrão Builder

A OS tem muitos atributos, e um construtor com todos eles ficaria confuso de usar. Por isso usamos o Builder.

- O Builder é uma classe static dentro da OrdemServico
- Cada método do Builder faz return this, e assim dá pra encadear as chamadas
- O build() chama o construtor da OrdemServico
- O construtor é private, então new OrdemServico() fora da classe não compila

```
public class OrdemServico {
    private Long codigoUnico;
    private String nomeCliente;
    private StatusOrdemServico status;

    private OrdemServico(Builder builder) {
        this.nomeCliente = builder.nomeCliente;
        this.status = builder.status;
    }

    public static Builder builder() {
        return new Builder();
    }

    public static class Builder {
        private String nomeCliente;
        private StatusOrdemServico status;

        public Builder nomeCliente(String nomeCliente) {
            this.nomeCliente = nomeCliente;
            return this;
        }

        public OrdemServico build() {
            return new OrdemServico(this);
        }
    }
}
```

DTOs e Mappers

A API não recebe mais a entidade direto. O controller recebe um DTO, e o Mapper converte esse DTO para a classe de modelo e depois de volta.

- Assim o cliente não manda campos como codigoUnico e datas, que quem gera é o sistema
- Os mappers têm dois métodos static: toEntity() e toDTO()
- O OrdemServicoMapper usa o Builder para criar a OS
- Criamos PecaDTO, ServicoDTO, OrdemServicoDTO, PedidoPecasDTO, ItemPedidoDTO e AtualizarStatusDTO

```
public class OrdemServicoMapper {

    public static OrdemServico toEntity(OrdemServicoDTO dto,
                                        List<Servico> servicos) {
        return OrdemServico.builder()
                .nomeCliente(dto.getNomeCliente())
                .placaVeiculo(dto.getPlacaVeiculo())
                .descricaoProblema(dto.getDescricaoProblema())
                .servicos(servicos)
                .status(StatusOrdemServico.ABERTO)
                .dataAbertura(LocalDateTime.now())
                .build();
    }

    public static OrdemServicoDTO toDTO(OrdemServico os) {
        OrdemServicoDTO dto = new OrdemServicoDTO();
        dto.setCodigoUnico(os.getCodigoUnico());
        dto.setNomeCliente(os.getNomeCliente());
        dto.setStatus(os.getStatus().name());
        dto.setValorTotal(os.getValorTotal());
        return dto;
    }
}
```

Rotas

US	Verbo	Rota	Status	Ação
US01	`POST`	`/api/ordens-servico`	201 / 400 / 404	Criar OS
US02	`PUT`	`/api/ordens-servico/{id}/status`	200 / 404	Mudar status da OS
US03	`POST`	`/api/pedidos`	201	Criar pedido
US04	`POST`	`/api/pedidos/{id}/itens`	200 / 404	Adicionar peças no pedido
US05	`PUT`	`/api/pedidos/{id}/status`	200 / 404	Mudar status do pedido

Também tem GET para listar e buscar por id nas duas rotas.

Diagrama de classes

Atualizamos o diagrama da OAT 1 com a OrdemServico, o Builder, o PedidoPecas, o ItemPedido, os enums de status, os DTOs e os Mappers.

O Builder aparece ligado à OrdemServico como classe interna. Os membros static estão sublinhados.

O diagrama inteiro está no repositório.

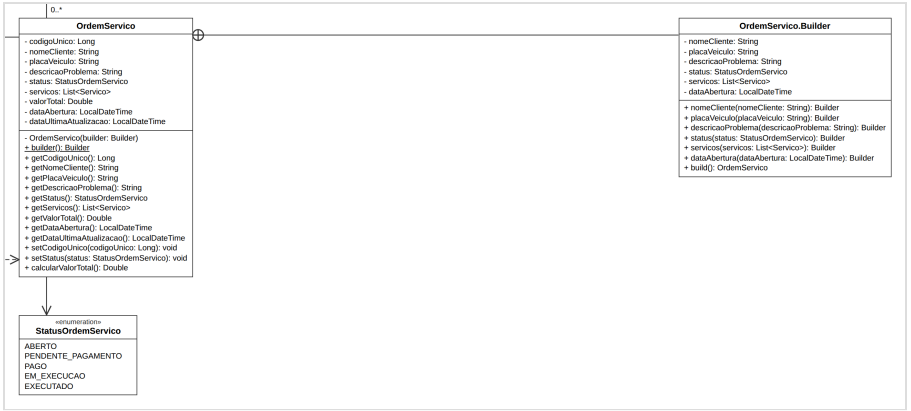
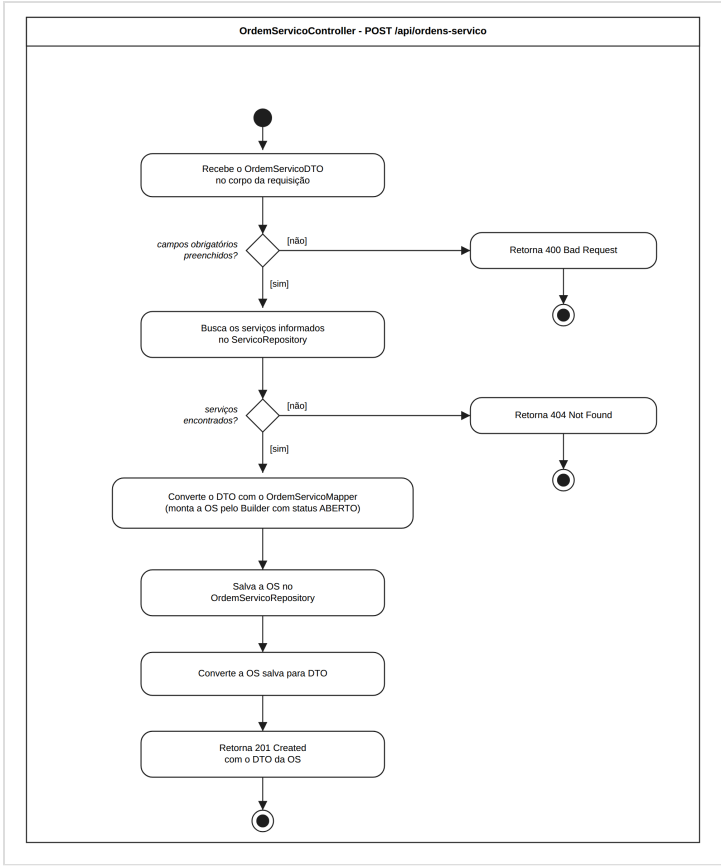


Diagrama de atividade

Escolhemos o POST /api/ordens-servico (US01), mostrando só a parte do controller.

- Se faltar algum campo obrigatório, retorna 400
- Se algum serviço informado não existir, retorna 404
- Se estiver tudo certo, o mapper monta a OS com o Builder, o repository salva e retorna 201 com o DTO



Considerações finais

- Com o Builder ficou mais fácil criar a OS
- Com os DTOs a API não mostra mais a entidade inteira
- A entidade associativa resolveu a quantidade de peças no pedido
- Falta ainda validar a ordem dos status (por exemplo, só executar a OS depois de paga)

`github.com/munizntc/mecaniQA-api-Vitoria`

Referências

- Conteúdo das aulas
- https://refactoring.guru/pt-br/design-patterns/builder
- https://docs.spring.io/spring-boot/